

Figure 2: A running application screen

```

    }
}

sub handle_submit_click{
    #Get modified values from all widgets
    my ($a_f, $fn, $cm_id);
    my $sf_rbt = $boulder->get_object("male_rbt");
    $a_f = 'F';
    $a_f = 'M' if($sf_rbt->get_active() == TRUE);
    $fn = $boulder->get_object("fn_ent")->get_text();
    $cm_id = $boulder->get_active();
    #Get other values and submit to database here
    $cm->hide() if(defined($cm));
}

```

In addition to the above code, we need to modify the `clear_page1()` function to handle the name change as well, by adding one line of code, as follows:

```
$boulder->get_object("srch_btn")->set_label("Search");
```

Status bar and menu entries

The status bar is used to show status messages to the user. To do this, the message should be pushed to the `StatusBar` widget, with a context ID to identify the context of the message. There can be multiple context IDs associated with a status bar; only the message which is at the top of the stack is displayed, irrespective of the context ID. We have already included a status bar in our application. To use it, we need to generate at least one context ID:

```

$ctx_id = $b->get_context_id("EMPLOYEE2"); #
This string can be anything

```

When you want to display a message, push the message onto the top of the stack, using the following code:

```
$b->push($ctx_id, "Message to display")
```

Popping the current top of the stack before pushing a new message is a good idea. Otherwise, when you pop the stack, an old message will be displayed. So, unless you want to preserve the old message, pop the message stack.

Handling menu entries

Finally, let's add a handler for the `File->Quit` menu entry. Double-click `File` in the top level window, and click on `Quit`. In the `Properties` pane, change the name to `quit_mni`. Add a handler for the `activate` signal, with the name `on_quit_mni_activate` and define the handler function in the Perl script:

```

sub on_quit_mni_activate{
    Gtk2->main_quit(); # quit the top window...
}

```

This completes our whirlwind tour of Glade, GTK and Perl. The accompanying code has incorporated all the topics we have discussed so far, with detailed comments. In the next chapter, we will retrace the steps using GTK, C and GLib. 

Authorised Training Partner

EnterpriseDB  

The Enterprise Postgres Company



redhat
CERTIFIED
SOLUTION PROVIDER

Authorized
Training Partner

Job Oriented Training for

Freshers

**LAMP Pro, Python,
Ruby on Rails, Perl,
Shell Scripting**



RHCE

Rs **25000 18500**
Training + Exam - Limited Offer

RHCVA

Red Hat Certified Virtualization
Administration

Linux For Software Tester

Selenium, Junit

Open Source Tool for
S/W Testing



Authorized
Training Partner

Zend Certification

ZCE - International Certification for PHP
from Zend Technologies - USA

Hostel Facility available
For our station students



Lynus Academy Pvt Ltd

1961-B, Vijaya Complex - 5th Floor
Asiad Colony, Anna Nagar - West
Chennai 600101

**044-42171278
9952966527**

contactus@lynusacademy.com
www.lynusacademy.com

By: Om Narasimhan

The author is a senior Linux kernel engineer at Oracle Corporation in the x86 systems group, Menlo Park, California.